id	key	value
1	FEE_RATE	0.05
2	FOO	bar

Instead of having a single row, you have a row for every setting. I, personally, like the singleton approach better since it's more "reliable." You just look at the table and know exactly what settings your project has. The key-value store is a bit "random." Any developer can insert any value without others knowing about it. Unless you maintain an enum or array with the possible keys. In the singleton model, if you need a new setting, you have to add a new column, with a type, and possibly with a default value as well. It's visible and well-documented. Works better for me. Of course, you can end up with a table that has 50+ columns. Which can be annoying.

So we have a `settings` table. The `fee_rate` column is cast into a `Percent` value object:

```
namespace App\ValueObjects;

class Percent
{
  public string $formattedValue;

  public function __construct(public float $value)
  {
    $this->formattedValue = number_format($this->value * 100, 2) . '%';
  }

  public static function make(float $value): self
  {
    return new static($value);
  }

  /**
   * Percent value is a value such as 15 or 99.93
   */
  public static function fromPercentValue(float $value): self
  {
    return new static($value / 100);
  }
}
```